PACIFIC INSTITUTE OF TECHNOLOGY — DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CS 460: Deep Learning & Neural Architectures

Assignment 2 Master Solution & Grading Rubric | Maximum Marks: 50.0

Q1 (12.0 Marks): (a) Explain the Adam optimizer update equations. Derive why bias correction terms (1 - beta_1^t) and (1 - beta_2^t) are necessary for the first and second moment estimates m_t and v_t when initialized at zero. (b) Contrast SGD with Nesterov Accelerated Gradient (NAG) vs Adam in non-convex loss landscapes with narrow ravines and saddle points.

Reference Model Solution:

(a) Adam Update Equations & Bias Correction Derivation: Adam computes exponential moving averages of past gradients (first moment m_t = beta_1 * m_{t-1} + (1 - beta_1) * g_t) and squared gradients (second moment v_t = beta_2 * v_{t-1} + (1 - beta_2) * g_t^2). When m_0 = 0, expanding m_t recursively yields m_t = (1 - beta_1) * sum_{i=1}^t beta_1^{t-i} * g_i. Taking expectations on both sides under the assumption that true expectation E[g_i] is stationary (E[g_i] = E[g_t]): E[m_t] = E[(1 - beta_1) * sum_{i=1}^t beta_1^{t-i} * g_i] = E[g_t] * (1 - beta_1) * sum_{i=1}^t beta_1^{t-i} = E[g_t] * (1 - beta_1) * ((1 - beta_1^t) / (1 - beta_1)) = E[g_t] * (1 - beta_1^t). Because (1 - beta_1^t) < 1, the raw estimator m_t is severely biased toward zero during early iterations (small t). Dividing m_t by (1 - beta_1^t) yields the unbiased estimator m_hat_t such that E[m_hat_t] = E[g_t]. Exactly the same derivation applies to the uncentered variance v_t, yielding v_hat_t = v_t / (1 - beta_2^t). (b) SGD+NAG vs Adam: In narrow ravines and ill-conditioned curvatures, standard SGD oscillates violently across steep walls with negligible progress along gentle slopes. NAG computes gradients at a look-ahead position (theta + beta * v), anticipating future slope changes and damping oscillations. However, NAG maintains a single global learning rate. Adam utilizes coordinate-wise adaptive learning rates by dividing by sqrt(v_hat_t) + eps, automatically scaling down steps along steep dimensions (large second moment) and accelerating steps along flat dimensions (small second moment), escaping plateaus and saddle points far more rapidly.

Required Key Concepts: *Adam first and second moment update formulas (m_t, v_t), Recursive expansion of m_t starting from zero initialization, Expectation proof showing E[m_t] = E[g_t] * (1 - beta_1^t), Unbiased estimator formulation m_hat_t = m_t / (1 - beta_1^t) and v_hat_t = v_t / (1 - beta_2^t), Comparison of coordinate-wise adaptive scaling vs Nesterov look-ahead momentum on ravines*

Rubric: *Full: Complete recursive expansion and algebraic derivation of (1 - beta_1^t), correct bias-corrected estimators, and sound comparative analysis of coordinate-wise scaling vs NAG momentum. | Partial: States Adam equations accurately but omits expectation proof for bias correction, or gives superficial NAG comparison.*

Q2 (13.0 Marks): (a) In the Transformer architecture, define the Scaled Dot-Product Attention mechanism. Mathematically prove why the dot product Q*K^T must be scaled by 1/sqrt(d_k), explaining its effect on the softmax gradient. (b) Explain the representational capacity of Multi-Head Attention over Single-Head Attention. Analyze the computational complexity comparison between self-attention O(n^2 * d) and recurrent layers O(n * d^2).

Reference Model Solution:

(a) Scaled Dot-Product Attention is defined as Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V, where Q, K are queries and keys of dimension d_k, and V is values of dimension d_v. Scaling factor derivation: Assume components of q and k are independent random variables with zero mean (E[q_i] = E[k_i] = 0) and unit variance (Var(q_i) = Var(k_i) = 1). The dot product is q . k = sum_{i=1}^{d_k} q_i * k_i. The expected value is E[q . k] = sum E[q_i * k_i] = 0. The variance is Var(q . k) = sum Var(q_i * k_i) = sum (Var(q_i) * Var(k_i)) = d_k * (1 * 1) = d_k. Thus, standard deviation of the dot product grows as sqrt(d_k). For large d_k, dot products grow substantially in magnitude, pushing the softmax function into regions with extremely small gradients (gradient saturation where d(softmax_i)/dz_j -> 0 when z_i >> z_j). Scaling by 1/sqrt(d_k) normalizes the variance back to Var((q . k) / sqrt(d_k)) = Var(q . k) / d_k = d_k / d_k = 1, ensuring stable gradient flow during backpropagation. (b) Multi-Head Attention vs Single-Head: Instead of performing a single attention function with d_model-dimensional keys, Multi-Head Attention linearly projects Q, K, V into h distinct lower-dimensional subspaces of dimension d_k = d_model / h. This allows the model to jointly attend to information from multiple representation subspaces at different positions (e.g., syntactic vs semantic relationships simultaneously). Single-head attention averages across these subspaces, losing fine-grained directional specialization. Complexity: Self-attention processes sequence of length n with dimension d in O(n^2 * d) time per layer with O(1) sequential path length. Recurrent layers (RNN/LSTM) require O(n * d^2) operations with O(n) sequential path length. When sequence length n < representation dimension d, self-attention is computationally faster and natively parallelizable across all tokens.

Required Key Concepts: *Scaled Dot-Product Attention formula Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V, Variance derivation of dot product proving Var(q . k) = d_k under unit variance assumption, Softmax gradient saturation critique (vanishing gradients for large input magnitudes), Multi-Head subspace projection rationale (attending to diverse linguistic/relational features), Complexity analysis O(n^2 * d) vs RNN O(n * d^2) and O(1) sequential execution path*

Rubric: *Full: Rigorous variance proof of Var(q.k)=d_k, clear explanation of softmax vanishing gradient prevention, multi-head subspace benefits, and complete complexity tradeoff. | Partial: States variance scaling intuitively without formal summation proof, or omits complexity analysis.*

Q3 (12.0 Marks): (a) Compare Batch Normalization (BatchNorm) and Layer Normalization (LayerNorm) regarding normalization axes, dependencies across batch size, and suitability for sequential models. (b) Explain Inverted Dropout. Derive why activations must be scaled by 1/(1 - p) during training and explain why no scaling is required during inference.

Reference Model Solution:

(a) BatchNorm vs LayerNorm: Batch Normalization computes the mean and variance across the batch dimension (N) for each individual channel/feature (C, H, W). Consequently, BatchNorm depends strictly on mini-batch statistics; its performance degrades severely when batch size is small (e.g., N < 16) or varies dynamically across steps, and it requires maintaining running population statistics (mu_running, sigma^2_running) for inference. Furthermore, in variable-length sequential/RNN/Transformer models, sentence lengths vary, making cross-batch statistics problematic. Layer Normalization normalizes across all feature dimensions (C, H, W) for each individual sample independently (across the channel/hidden axis). Thus, LayerNorm has zero cross-sample dependencies, behaves identically at batch size 1, requires no running statistics across iterations, and is ideal for sequential models (Transformers, RNNs). (b) Inverted Dropout Derivation: In standard dropout with drop probability p, each neuron activation a is multiplied by a Bernoulli random mask r ~ Bernoulli(1 - p). During training, the expected activation is E[r * a] = (1 - p) * a. In naive dropout, test-time activations must be scaled by (1 - p) to match this expectation: a_test = (1 - p) * a. Inverted Dropout applies scaling directly during training by setting a_train = (r * a) / (1 - p). Expected value during training: E[a_train] = E[(r * a) / (1 - p)] = ((1 - p) * a) / (1 - p) = a. Because training expectation is already normalized to match the unmasked activation a, NO mathematical adjustments or scaling operations are needed at test time (a_test = a), dramatically accelerating inference and simplifying model deployment.

Required Key Concepts: *BatchNorm normalizes across batch axis N; LayerNorm normalizes across feature axis per sample, BatchNorm sensitivity to small batch size and requirement of running population statistics, LayerNorm independence from batch size and suitability for variable-length sequence models, Inverted Dropout training scaling factor 1 / (1 - p), Expectation proof E[a_train] = a showing why test-time requires zero scaling*

Rubric: *Full: Clear diagrammatic contrast of normalization axes, impact of mini-batch size, mathematical derivation of inverted dropout expectation invariance, and inference simplicity. | Partial: Contrasts BatchNorm and LayerNorm correctly but lacks mathematical expectation proof for inverted dropout.*

Q4 (13.0 Marks): (a) For deep Residual Networks (ResNets), state the identity mapping formulation x_{l+1} = x_l + F(x_l, W_l). Derive the gradient flow equation dE/dx_l and mathematically explain why vanishing gradients are avoided even as network depth exceeds 100 layers. (b) Contrast standard residual blocks with Pre-Activation Residual Blocks.

Reference Model Solution:

(a) ResNet Gradient Flow Derivation: For any two layers l and L where L > l, the recursive identity formulation x_{l+1} = x_l + F(x_l) can be expanded telescopically to: x_L = x_l + sum_{i=l}^{L-1} F(x_i, W_i). Let E be the scalar loss function. Applying the multivariable chain rule to compute the gradient with respect to the earlier layer representation x_l: dE/dx_l = (dE/dx_L) * (dx_L/dx_l) = (dE/dx_L) * (d/dx_l (x_l + sum_{i=l}^{L-1} F(x_i, W_i))) = (dE/dx_L) * (I + d/dx_l sum_{i=l}^{L-1} F(x_i, W_i)), where I is the identity matrix. Why Vanishing Gradients are Avoided: The gradient dE/dx_l decomposes into two additive terms: (dE/dx_L) * I (direct, uninterrupted gradient highway from output layer L straight to input layer l) and (dE/dx_L) * (d/dx_l sum F). In traditional feedforward networks without skip connections, the gradient is a pure product of Jacobian matrices prod_{i=l}^L W_i, which exponentially decays to zero if singular values are < 1. In ResNets, the additive identity term I ensures that even if the weight Jacobian sum d/dx_l sum F decays toward zero, the gradient dE/dx_l does NOT vanish because it retains the non-vanishing component (dE/dx_L) * I. (b) Pre-Activation vs Post-Activation: In standard (post-activation) ResNet blocks, the addition is followed by Batch Normalization and ReLU: x_{l+1} = ReLU(BN(x_l + F(x_l))). This non-linear operation alters the clean identity highway x_l -> x_{l+1}. In Pre-Activation ResNets (He et al., 2016), BN and ReLU are moved inside the residual branch prior to weight multiplication: x_{l+1} = x_l + F(ReLU(BN(x_l))). This preserves a 100% clean identity path across the entire network, leading to easier optimization and lower test error for extremely deep networks (e.g. ResNet-1001).

Required Key Concepts: *Telescopic expansion of residual layers x_L = x_l + sum F(x_i), Chain rule gradient formulation dE/dx_l = dE/dx_L * (I + d/dx_l sum F), Additive identity term I acts as an uninterrupted gradient highway, Contrast with multiplicative Jacobian chain prod W_i causing exponential vanishing gradients, Pre-activation architectural restructuring preserving unhampered identity flow*

Rubric: *Full: Rigorous telescopic expansion, exact chain rule derivation showing identity term I, clear proof of vanishing gradient prevention, and pre-activation comparison. | Partial: Derives gradient formula but misses telescopic expansion or gives incomplete explanation of why identity term prevents vanishing.*